KIROVAULT

Rust Backend & Filesystem Engine

Define safe native commands for browsing and managing real Windows files while keeping the UI responsive.

DOCUMENT PURPOSE

This standalone companion document complements the KiroVault Desktop App Blueprint. It follows the Windows-first, local-first product direction. Recommendations are proposed guidance, not claims that these features already exist.

Contents and implementation guidance

Suggested modules

• commands/filesystem.rs: list, stat, create, rename, copy, move, delete, open-with and reveal.

• commands/indexing.rs: start/pause/resume/cancel scans, progress and skipped-path reporting.

• commands/thumbnails.rs: request/cancel, cache lookup/invalidation and bounded worker queue.

• commands/metadata.rs: safe image/audio/video metadata readers.

• commands/search.rs: name/path/type/date/size queries and optional content search.

• db/: migrations, prepared queries, indexes and integrity checks.

Filesystem operation contract

• Validate names and destination paths; detect collisions before mutation.

• Copy/move conflict choices: Skip, Replace, Keep both or Cancel.

• Prefer Recycle Bin for normal delete where supported; permanent deletion requires explicit choice.

• Return per-item outcomes for bulk operations and report partial failures accurately.

• Reconcile actual filesystem state after interruption before resuming jobs.

Path safety

• Canonicalize paths where appropriate and verify access at operation time.

• Handle symlinks/junctions deliberately; do not assume lexical path prefixes prove containment.

• Respect Windows permissions; never elevate privileges automatically to bypass access failures.

• Treat filenames and metadata as untrusted input; avoid shell interpolation.

Background workers

• Bound concurrency, memory, input bytes and execution time.

• Keep scan and thumbnail queues separate; allow cancellation where practical.

• Persist scan checkpoints and show inaccessible/skipped paths.

• Folder browsing remains available during long-running scans.

Rust tests

• Test invalid characters, long paths, Unicode names, collisions and cross-drive operations.

• Test inaccessible folders, stale paths, disconnected removable drives and junctions.

• Verify permanent delete confirmation and Recycle Bin behavior.

• Inject worker errors and confirm partial failures are reported.

References

• Rust Book: https://doc.rust-lang.org/book/

• Tauri v2 commands: https://v2.tauri.app/develop/calling-rust/

- Tauri filesystem plugin: https://v2.tauri.app/plugin/file-system/
- Microsoft Windows file access: https://learn.microsoft.com/en-us/windows/win32/fileio/file-security-and-access-rights

Reference note: external documentation may change. Verify current versions, licensing, APIs and platform prerequisites during implementation.